the official documentation for an extensive list of predicates available in Spring Cloud Gateway: https://spring.io/projects/spring-cloud-gateway.

Defining routes with the Java/Kotlin DSL

Spring Cloud Gateway is a very flexible project that lets you configure routes the way that best suits your needs. Here you have configured routes in a property file (application.yml or application.properties), but there's also a DSL available for configuring routes programmatically in Java or Kotlin. Future versions of the project will also implement a feature to fetch the route configuration from a data source using Spring Data.

How you use it is up to you. Putting routes in configuration properties gives you the chance to customize them easily depending on the environment and to update them at runtime without the need to rebuild and redeploy the application. For example, you would get those benefits when using Spring Cloud Config Server. On the other hand, the DSL for Java and Kotlin lets you define more complex routes. Configuration properties allow you to combine different predicates with an *AND* logical operator only. The DSL also enables you to use other logical operators like *OR* and *NOT*.

Let's verify that it works as intended. We'll use Docker to run the downstream services and PostgreSQL, whereas we'll run Edge Service locally on the JVM to make it more efficient to work with, since we are actively implementing the application.

First, we need both Catalog Service and Order Service up and running. From each project's root folder, run `./gradlew bootBuildImage` to package them as container images. Then start them via Docker Compose. Open a Terminal window, navigate to the folder where your docker-compose.yml file is located (polar-deployment/docker), and run the following command:

```
$ docker-compose up -d catalog-service order-service
```